O'REILLY®
Report

# Agentic AI Data Architectures

## How Distributed SQL Unifies Enterprise Scale and AI-Native Application Design

**Blaize Stewart & Ed Huang**

# PingCAP

# Agentic AI Data Architectures

## How Distributed SQL Unifies Enterprise Scale and AI-Native Application Design

**Blaize Stewart and Ed Huang**

## Agentic AI Data Architectures

by Blaize Stewart and Ed Huang



Published by O’Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O’Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*https://oreilly.com*). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisitions Editor: Aaron Black

Development Editor: Jeff Bleiel

Production Editor: Beth Kelly

Copyeditor: Penelope Perkins

Proofreader: O’Reilly Media, Inc.

Cover Designer: Karen Montgomery

Cover Illustrator: Susan Brown

Interior Designer: David Futato

Interior Illustrator: Kate Dullea

January 2026: First Edition

## Revision History for the First Edition

- 2026-01-15: First Release

979-8-341-66783-9

[LSI]

# Chapter 1. What Is Agentic AI and Why Memory Matters

If generative AI chat is a recipe book, then agentic AI is a personal chef. A recipe book offers countless instructions, but it cannot decide what meal makes sense for you today, nor can it adapt when your pantry is missing key ingredients. A chef, by contrast, knows your preferences, remembers what you liked last week, and keeps track of what is in the fridge and what has gone bad. The chef's value is greater than just preparing a single meal, because the chef draws on memory and experience to provide consistency and care over time.

This is the same distinction emerging in artificial intelligence. Generative AI is celebrated for producing text, answering questions, or automating routine tasks, just as a recipe book is valued for its collection of dishes. Yet these are isolated outputs. Agentic AI represents the move toward systems that act like a chef that is able to plan, adapt, and carry forward knowledge from one interaction to the next.

For this to be possible, memory is the key. Without it, a chef will forget your allergies, serve the same dish every day, or waste ingredients by overlooking what is already on hand. Likewise, AI agents without memory lose context, repeat mistakes, and fail to act beyond the immediate request. Building memory into AI is not optional. It is what enables persistence, adaptation, and continuity. In technical terms, agentic AI refers to systems that combine a large language model (LLM) with structured components such as memory stores, retrieval mechanisms, and planning modules. These systems integrate past interactions, data retrieval, and decision making to act coherently over time rather than simply respond to prompts. In other words, memory transforms AI from a reactive generator into an adaptive agent. For agents to act coherently at scale, their memories must be consistent, accessible, and resilient across distributed systems. This is where

distributed SQL becomes crucial. Agentic AI depends on a data infrastructure that can synchronize knowledge, preserve state, and ensure reliability even as demands grow. Chapter 2 explores how distributed SQL provides this foundation; but for now, we will focus on understanding agentic AI and what problems it creates for memory.

## What Makes AI "Agentic"?

As its name implies, the heart of agentic AI is *agency*. Agency is the capacity to act with a degree of autonomy on behalf of a user or a system. It implies intelligent response, but to accomplish these responses, agentic systems need to perceive, reason, act, and learn.[1] A regular machine learning model is static and task-specific. It takes input, produces output, and stops. It does not decide what to do next or remember why it did something before. For example, a model might classify an image, generate text, or predict a value, but it does not act beyond that single function. Agentic AI builds on these more static models by incorporating memory. Agentic systems are composed of four basic ideas:

*Perceive*

> An agent draws on signals from multiple sources such as text input, APIs, databases, sensors, or prior memory.

*Reason*

> It evaluates these inputs against objectives and constraints, then decides what should happen next.

*Act*

> Agents can act beyond the conversation itself, whether by triggering workflows, invoking external systems, or coordinating subtasks, so that progress toward a goal occurs without continuous human prompting.

*Learn*

> Agentic AI continuously improves by ingesting feedback, which forms a feedback loop that refines future performance and decision making.

This perceive/reason/act/learn loop is where agentic AI deviates from a chat system like ChatGPT. ChatGPT is primarily reactive: it produces fluent, useful responses within the boundaries of a conversation, but it does not carry objectives forward once the exchange ends. An agent, by contrast, retains goals and acts on them, even across multiple steps and systems. ChatGPT can describe how to book a flight; an agentic AI can actually search flight options, compare them against constraints, and make a booking. The difference is between a conversational assistant and an autonomous actor in both information and execution.

# The Limits of Models

Enterprise AI is no longer limited by model size, but rather by how effectively memory is used. The real challenge lies not in computational power or parameter counts, but in integrating, organizing, and applying knowledge with continuity. The question is whether systems can recall the right information at the right time and maintain coherence across tasks, contexts, and interactions. The future of productive AI will be defined less by ever-larger models and more by unified, adaptive, and reliable memory architectures.

## Diminishing Returns on Model Size

For enterprise applications of agentic AI, the real bottleneck is not the raw size of the underlying model but its ability to remember, retrieve, and reuse the right information at the right time.

*Neural scaling laws* are empirical power-law relationships that describe how a neural network's performance (e.g., test loss or error rate) improves as you scale critical resources like number of parameters, training dataset size, and computational budget. In essence, these laws show that

performance improves predictably as you scale these factors according to specific mathematical patterns rather than linearly.

But there is a catch: returns diminish as models get bigger. Going from a small model to a medium one gives a huge jump in accuracy. Going from a very large model to an even larger one gives only a small improvement, despite the costs skyrocketing. A 70-billion parameter model trained on outdated or shallow data will perform worse than a smaller model that has access to a broad, high-quality, and timely dataset. The famous Chinchilla model and subsequent study confirmed this by showing that a mid-sized model trained on much more data outperformed much larger models trained on less.[2] The insight was clear: better knowledge, not bigger models, makes all the difference.

## The Importance of Context

Research further shows that in-context information, supplied through prompts, is treated differently by LLMs than knowledge embedded in their weights. For example, transformers often use context to generalize by analogy or example, while their internal parameters encode more abstract, rule-like patterns. This distinction uncovers the power of external context provided at inference time. It can shape reasoning and guide behavior in ways static weights cannot.[3]

Of course, reliance on external knowledge introduces its own challenges. LLMs may place undue trust in retrieved material, even when it is incomplete or misleading. To mitigate this, researchers have developed confidence-aware strategies that help models balance what they “know” internally with what they are given externally. The goal is to ensure that models rely on external sources when they are reliable, while avoiding overcommitment to inaccurate or low-quality content. In systems like ChatGPT, “context” often means little more than the immediate chat history supplied in a single session. Agentic AI, however, must take on more than that: it needs continuity across tasks, awareness of prior outcomes, and the

ability to integrate signals from diverse sources beyond a single thread of text.

This broader challenge is what some are beginning to call *context engineering*, which is the deliberate design of how context is created, maintained, and applied to shape reasoning.[4] Within this view, "memory" becomes the durable substrate of context, providing more than just raw recall of what has been said. Memory is also the structured record of what was known, when it was known, and why it mattered for action. In essence, the weight of evidence shows relying on external content supplied at inference time from up-to-date, relevant sources tends to yield more reliable and contextually accurate outputs than depending solely on what's embedded in a model's internal weights.[5]

## Memory: Persistence Across Time

Context alone is insufficient if it disappears at the end of a session, which is why memory becomes critical. Stateless models treat every prompt as if it exists in isolation, even when they simulate continuity, such as a conversational window in an application (historically, like ChatGPT): once the conversation ends, that state is lost. The result is a brittle form of interaction that cannot build cumulative knowledge, highlighting the need for memory beyond the chat itself.

An agentic AI system, by contrast, can recall prior decisions, user preferences, constraints, and intermediate outputs. It can carry these outputs forward into future interactions and adapt its reasoning as new information arises.

### Short-Term Memory

Short-term, or working, memory refers to the system's ability to hold and use information within the scope of a single task or conversation. It allows the AI to keep track of what has just been said or done so that the interaction feels coherent. For example, when a user asks a travel-planning

agent about flights to Paris and then follows up with “make that London instead,” working memory ensures that “that” refers to the flight search rather than requiring the user to restate the entire request. Without this form of memory, interactions would reset at every turn, forcing users to repeat themselves.

## Long-Term Memory

Long-term memory extends persistence beyond the boundaries of a single session. This enables the system to retain knowledge and preferences over days, weeks, or months. For instance, an enterprise support agent might remember that a customer has previously struggled with software licensing issues and proactively check for those problems in future interactions. Or a personal health agent might track ongoing dietary habits over time, providing tailored recommendations based on accumulated history. This kind of memory moves the system closer to acting like a trusted collaborator rather than a disposable interface.

## Episodic Memory

Episodic memory captures sequences of events and their outcomes, allowing the agent to reflect on what worked and what failed. This is particularly valuable for agents that must plan and adapt. For example, a supply-chain management agent might record that rerouting shipments through a particular hub consistently led to delays. By remembering not just the fact but the sequence of actions and results, the system can learn from experience, avoid repeating errors, and refine strategies for future tasks.

Episodic memory thus gives the agent the ability to treat prior attempts as lessons rather than one-off events. This persistence transforms the agent into a system that not only reacts but learns and evolves, bringing forward accumulated experience much like a human collaborator would.

But lessons learned and experiences accumulated from memory are only as useful as the systems that can store, retrieve, and apply them consistently. Without the right underlying storage systems, memory becomes

fragmented, inaccessible, or too slow to influence real-time decisions. This is where the limits of traditional enterprise data systems become evident.

## Why Traditional Data Architecture Falls Short

As agentic AI matures, one of the biggest hurdles is data infrastructure. The very systems that have underpinned enterprise computing for decades were built for static queries, predictable loads, and narrow definitions of state. Agentic AI, by contrast, thrives on fluid reasoning across persisted memory and real-time access to knowledge. This fundamental mismatch makes it necessary to examine why traditional data infrastructure cannot keep pace, and why new architectures are required to support continuity, adaptability, and scale in the agentic era.

### How AI Development Outpaced Data Architecture

Large language model adoption began with prompt engineering, where users shaped inputs to guide stateless systems; but relying on context as the only form of memory quickly revealed its limits. This limitation drove the rise of agentic AI, which moves beyond single prompts by decomposing tasks, orchestrating steps, and maintaining continuity, thus enabling workflows like research assistance without constant user input. Multiagent ecosystems extend this further, allowing specialized agents to collaborate like teams. Yet this collaboration introduces new challenges from fragmented data across relational databases, knowledge bases, and vector stores, which can lead to latency, inconsistency, and stale information.

While models and frameworks have advanced, enterprise infrastructure remains oriented around batch jobs and predictable queries, leaving a gap between agentic AI’s potential and the continuity, integration, and adaptability its memory systems require.

### The Misalignment of Stateless Infrastructure and

### Memory

Multiagent systems introduced the need for memory, but there is significant friction between how traditional databases and agentic AI manage state. Conventional systems excel at transactional state by recording discrete, consistent snapshots of events such as financial transactions or account updates. This kind of state is reliable but static, treating each entry as an isolated photograph in a camera roll.

Agentic AI, by contrast, depends on evolving state that is a continuous sequence like a film reel, where each frame is shaped by what came before and informs what comes next. Interrupting or discarding frames breaks continuity, undermining the reasoning process.

While stateless, transactional systems can approximate continuity with mechanisms like session tables, caches, or orchestration layers, these are brittle work-arounds not designed for inference-driven workloads. The result shows up in practical failures: conversational resets, planning agents that lose track of subtasks, or bots that give inconsistent responses because context was not preserved. In short, traditional databases provide excellent static records, but they cannot natively sustain the adaptive, fluid memory that agentic AI requires.

## Toward Memory as Infrastructure

If memory is what grants agents the capacity to act with persistence and purpose, then infrastructure is what determines whether that capacity can endure. Memory cannot be left to chance or patched together from tools that were never intended to sustain it. It must rest on foundations specifically designed to preserve continuity, integrate context, and adapt as objectives change. Just as no chef can serve reliably in a kitchen without order, equipment, and flow, no agent can serve effectively without an environment built for memory.

The question is not whether memory matters, but how infrastructure must evolve to make it dependable. The next step is to examine what goes wrong

when traditional systems are forced into this role, and how new designs can intentionally support the continuity, adaptability, and scale that agentic AI requires.

---

1 Erik Pounds, "What Is Agentic AI?", NVIDIA (blog), October 22, 2024.

2 Jordan Hoffmann et al., "Training Compute-Optimal Large Language Models", arXiv (preprint), March 29 2022.

3 Bryan Chan et al., "Toward Understanding In-Context Vs. In-Weight Learning", paper presented at ICLR 2025, January 22, 2025.

4 Phil Schmid, "The New Skill in AI Is Not Prompting, It's Context Engineering", blog, June 30, 2025.

5 Aakash Gupta, "Context Engineering: The Evolution Beyond Prompt Engineering That's Revolutionizing AI Agent Development", *Medium*, July 16, 2025.

# Chapter 2. Memory as Infrastructure

Chapter 1 established that perhaps the biggest obstacle to enterprise AI is not access to models but access to data. And for agentic AI, part of that data is memory. Memory spans contracts, logs, events, and metrics, forming the substrate of every decision. When unified, it enables coordination; when fragmented, intelligence stalls. Treating memory as infrastructure shifts AI from experimental to operational capability.

Memory gives agents a stable way to find facts, interpret meaning, carry context forward, and preserve a record of what was known, when it was known, and why it mattered. Seen this way, memory becomes a first-class property of architecture. A memory infrastructure must be built with the same rigor as networks or storage systems that are shared, reliable, and governed. With that foundation in place, the broader AI stack stops pulling against itself and begins to function as a cohesive system.

The next step is to confront the problems caused by fragmented data ecosystems and see why distributed SQL offers the path toward the consistency and scale that memory-driven AI requires.

## The Fragmented Data Ecosystem

Enterprises rely on data in many forms, from rigidly structured transactions to loosely organized documents and time-sensitive signals unfolding across events. Each type of data carries distinct strengths and limitations, yet all must be integrated into a coherent substrate for agents to operate with continuity. Understanding this requires a closer examination of the types of data—structured, unstructured, and temporal—that compose the enterprise ecosystem.

## Structured Data

*Structured data* (comprising facts, transactions, and metadata) anchors agentic AI in reliable, auditable, and precise truths. Currently, only about 20% of enterprise data is structured,[1] yet it plays an outsized role in training machine learning models and enabling classification, regression, and predictive accuracy. Its well-defined format allows agents to ground reasoning with clarity. A transactional record confirms that an event occurred at a given time, while a sensor threshold breach signals a precise condition. In essence, structured data functions like immutable digital laws, necessary for triggering actions, ensuring compliance, and maintaining consistency in AI decision making.[2]

## Unstructured Data

*Unstructured data* (spanning documents, conversations, logs, images, and more) represents the vast majority of organizational knowledge. While structured data offers clarity, unstructured data holds nuance, such as tone, intent, and context. Embedding and interpreting these inputs unlocks deeper meaning, enabling agents to comprehend not only what occurred, but why it matters.

Despite its volume and importance, many organizations struggle to leverage unstructured data effectively. Effectively processing this data through semantic embeddings and retrieval systems is essential for agents to be able to resolve ambiguity, bridge context gaps, and adapt decisions to subtle human and institutional nuances.

## Temporal Continuity

*Temporal continuity* enables agentic AI to recall, adapt, and act over time. Three key mechanisms make this possible:

*Episodic memory*

> Empowers agents to store and retrieve specific past events, enhancing decision making, planning, and personalization, akin to human memory

*State tracking*

: Maintains continuity across dialogue turns or operational steps, ensuring agents preserve goals and context even when sessions shift

*Time-sensitive retrieval*

: Filters knowledge to surface what's current and relevant, minimizing reliance on outdated information

Still, episodic memory also introduces new risks, such as unpredictability, retention of sensitive knowledge, and manipulation vectors, which researchers are actively investigating to guide safe deployment.[3]

Structured, unstructured, and temporal data form the raw material of agentic AI. Each type plays a critical role, but their differences are mirrored in the systems built to store them, namely relational databases for structured facts, content repositories for documents, vector stores for embeddings, and specialized pipelines for time-sensitive events. These fragmented memory stacks evolved independently, optimized for narrow workloads rather than cross-cutting memory. When agents must reason fluidly across all three, the result is not seamless intelligence but friction that exposes points of failure.

## Common Failure Modes with Fragmented Memory Stacks

The very features that make these agentic AI systems powerful with multimodal knowledge flows, iterative reasoning, and adaptive orchestration also create new vulnerabilities. Memory fragments across silos, reasoning slows under its own weight, and operations strain against immature tooling. These are not bugs in a single component but systemic tensions that arise when intelligence depends on many moving parts working together in real time. Without coherence, accessibility, and control, agentic AI risks sounding intelligent while acting unpredictably.

In agentic AI systems, knowledge flows in from many directions. There is structured data, such as SQL tables, alongside unstructured embeddings in vector silos derived from images, videos, audio, and text. Left unmanaged, these streams create an environment where agents reason without clear or consistent signposts.

## The Latency Trap in Agentic AI

Multihop reasoning, where agents plan, retrieve, act, and reflect in cycles, is central to agentic AI, but it comes with trade-offs. Each loop adds latency that can outweigh the speed of traditional pipelines, making persistence feel like delay. The problem worsens when memory spans multiple systems, such as vector stores, SQL databases, and caches, which introduce costly handoffs and inconsistent results. Queries that traverse several backends accumulate latency while raising the risk of mismatches.

Developers often attempt to mitigate these problems with techniques such as adaptive looping, cascading models, partial parallelization, or aggressive caching.[4] Each of these can shave off some latency or improve precision in specific contexts, but they do so by layering more orchestration atop an already complex stack. The result is a system that may perform better in the short term but is increasingly fragile in the long run. Every handoff (deciding which model to call, which store to query, or when to stop iterating) becomes a new point of failure.

## Scaling Limits of Legacy Systems

Legacy systems, like monolithic databases, on-premises servers, or early-generation cloud architectures, often falter under the unique demands of inference-driven workloads. Traditional infrastructures are designed for predictable query volumes and batch processing, where scaling can be planned in advance. Agentic AI, however, introduces spiky workloads because of the way inferencing works. An agent may suddenly need to query thousands of records, apply semantic filters, and join results in real time, only to sit idle moments later.

Conventional, inelastic architectures struggle with this volatility. They cannot elastically scale to meet sudden peaks, nor can they prioritize the freshness of data needed for inference. This leaves organizations choosing between throughput and accuracy, typically sacrificing one for the other.

The tension between throughput and freshness is particularly acute. High throughput systems can serve many queries quickly but often rely on stale caches or relaxed consistency. Systems tuned for freshness, by contrast, may slow under load, introducing bottlenecks. For agents, this trade-off is unacceptable. Reasoning quality depends on both speed and accuracy. A fraud detection agent, for example, must analyze transactions as they occur, not hours later, and it must do so without collapsing under the weight of millions of queries. Meeting this dual requirement pushes beyond what legacy systems were designed to deliver.

## Operational Complexity: Orchestration Fragility, Monitoring Blind Spots

Running agentic AI is hard because the ecosystem for managing these systems is still immature. Unlike microservices, which have the benefit of Kubernetes and other well-established orchestration frameworks, agentic systems lack a comparable “Kubernetes for agents” (although there have been some attempts to bridge this gap with things like LLM meshes), but that only answers the computational side, not the storage.[5] Tooling for deployment, scaling, and monitoring exists, but it is fragmented, experimental, and often stitched together by hand. This makes it harder to ensure predictable performance as systems grow.

For example, tracing why an agent made a particular decision is notoriously difficult. Multiagent environments follow dynamic reasoning paths and nondeterministic loops, which means the same request can lead to different actions at different times. For engineers, this complicates debugging. For businesses, it complicates auditing and compliance.

Operational fragility also becomes a concern. A timeout in a single service call or a failure in one agent can cascade across the system, thereby

interrupting workflows and creating broad outages.

Finally, monitoring itself is hampered by blind spots. Standard practices for logging, tracing, and performance measurement are still emerging for agentic AI. Without them, organizations may not detect subtle issues, such as reasoning drift, growing hallucination rates, or unintended behaviors, until they escalate into serious problems.

Agentic AI reframes the challenge of artificial intelligence from being primarily about model capability to being fundamentally about systems design. If agents are to perceive, deliberate, and execute in ways that approach autonomy, then the infrastructures beneath them must evolve as well. This is no longer a question of marginal gains in prompt design or incremental improvements in model output, but of building data architectures that can sustain memory, unify diverse forms of retrieval, and adapt to the volatility of inference-driven workloads.

The mandate is clear. The success of agentic AI will hinge less on what the models themselves can generate and more on whether organizations can provide the foundations that allow those models to act with continuity, coherence, and trustworthiness.

## The Case for Distributed SQL

The shortcomings of traditional infrastructure show the need for a different foundation that is not constrained by rigid transactional models, fragmented retrieval pipelines, or brittle scaling assumptions. Distributed SQL has emerged as one of the most promising solutions to avoid these problems. It extends the proven strengths of relational databases into a distributed, multimodal context, thus offering a path toward unifying retrieval, preserving continuity, and sustaining the workloads that define agentic AI.

Distributed SQL refers to a new class of relational databases that preserve the familiar SQL model, including tables, schemas, and ACID (atomicity, consistency, isolation, and durability) transactions, while spreading the data and computation across multiple nodes, oftentimes geographically

separated. This architecture allows the system to scale horizontally, maintain strong consistency, and deliver high availability, combining the reliability of traditional relational databases with the elasticity and fault tolerance of cloud native systems. Distributed SQL platforms expose a single SQL interface while running across many servers or nodes, maintaining ACID guarantees and strong consistency. To achieve this, data is automatically partitioned and replicated, with consensus protocols (such as Raft) ensuring agreement across nodes.

This design enables horizontal scalability, high availability, and fault tolerance, allowing the system to handle failures gracefully and serve geographically distributed workloads. In practice, distributed SQL delivers the familiar power of complex relational queries and transactional integrity, but with the resilience and elasticity required for modern, global applications.

When a SQL database is distributed across multiple nodes, it must contend with the realities of network partitioning. The CAP theorem states that in the presence of a partition, a distributed system can guarantee at most two of three properties: consistency (every read reflects the most recent write), availability (the system continues to operate and respond to every request), and partition tolerance (the system continues functioning even when parts of the network are unreachable). This trade-off forces designers to decide whether to prioritize strict correctness by temporarily denying some requests or to remain operational by allowing access to potentially stale data.

Distributed SQL systems typically favor consistency over availability, using consensus protocols to safeguard data integrity even if responsiveness drops during failures. For many agentic AI workloads, this consistency-first approach is vital, as agents often depend on synchronized memory and coherent state to coordinate reasoning and make reliable decisions. However, not all agents require strict consistency. Some thrive on perfect coherence, while others benefit from immediacy. Distributed SQL provides the flexibility to navigate this trade-off, thus making it a strong foundation for a wide range of agentic applications.

## A Unified Retrieval Foundation

As agents advance to handle more complex tasks, the long-standing separation between transactional and semantic data systems is becoming a weakness. Traditionally, relational databases managed structured information such as financial transactions and customer records, while vector stores were added to support semantic retrieval of unstructured data like documents and embeddings. This split creates fragmentation, forcing agents to coordinate across multiple backends and introducing both latency and inconsistency.

Distributed SQL does not inherently provide vector search, but many platforms are beginning to integrate it either natively or through extensions. This development is significant. With vector capabilities, distributed SQL becomes a unified foundation where transactional accuracy and semantic similarity searches operate side by side.

## Declarative Power of SQL for Agentic Retrieval

SQL might seem antiquated, but SQL's declarative nature remains one of the most powerful assets in enterprise computing. Distributed SQL extends this familiar paradigm into the agentic era by enabling agents to make both transactional queries and, when vector search is incorporated, semantic similarity searches in a unified syntax. This fusion enables enterprises to leverage their existing governance, tooling, and workforce expertise while also unlocking new forms of AI-native retrieval.

For agents, the combination of declarative precision and semantic flexibility is essential because they can filter, join, and aggregate with accuracy, while simultaneously reasoning about meaning and context. Distributed SQL preserves the strengths of relational querying while adapting it to the needs of agentic AI by creating an infrastructure that remains both forward-compatible and grounded in decades of proven practice.

### Elasticity

Agentic AI workloads are inherently volatile. Distributed SQL systems are designed to meet spiky demand by scaling horizontally across clusters of nodes to provide capacity when demand spikes and retracting when workloads diminish. This elasticity makes retrieval remain performant even under surging inference-driven demand. Organizations may support many agents acting concurrently without sacrificing speed or reliability.

## With Agentic Apps, Distributed SQL Wins

AI agents now autonomously plan workflows, write code, generate content, test ideas, mutate data, and coordinate with other agents without human intervention. These applications behave less like static programs and more like living systems that are adaptive, iterative, and constantly evolving. They create new components, spawn subagents, alter schemas, and maintain long-running state as they pursue objectives.

This shift creates extraordinary demands on the underlying data layer that traditional databases were never designed to handle. And this is precisely where distributed SQL wins.

Agentic systems rarely work alone. A single high-level request can produce cascades of autonomous tasks with dozens or hundreds of agents working in parallel while each one maintains an intermediate context, generates a temporary state, or builds its own experimental “branch” of the task.

Distributed SQL systems thrive here because they can create lightweight, isolated data environments on demand. Their cloud native, decoupled architectures allow databases or logical clusters to appear and disappear almost instantly, enabling agents to branch, explore, and terminate their work without the overhead of manual database management. In effect, distributed SQL mirrors the agents’ own behavior: it is elastic, opportunistic, and capable of scaling horizontally at a moment’s notice.

Another defining characteristic of agentic applications is their continuous reshaping of schemas. They create new tables, modify existing fields,

generate semantic structures, or introduce novel indexes as they refine their understanding of a task. Modern implementations support online schema evolution, expansive metadata catalogues, and instant snapshotting or branching of data states. This allows agents to fork entire data environments, run experiments safely in isolation, evaluate the outcomes, and roll back or merge changes without disrupting the primary path of execution. It gives them a laboratory in which to test ideas continuously.

The workload patterns of agentic systems are chaotic by nature. An agent may sit idle for long periods and then, based on a triggering event, surge into a storm of activity involving thousands of read and write operations. Groups of agents may synchronize around shared objectives, producing sudden, overwhelming spikes in load. Distributed SQL platforms handle volatility with ease. Their distributed compute layers expand and contract fluidly, absorbing bursts of demand and redistributing work across nodes without human intervention. They transform unpredictability from a liability into an expected operating condition.

There is also an economic dimension to this story. Agentic systems are autonomous in that they generate their own tasks and queries. Distributed SQL platforms, however, typically include granular usage accounting, transparent metering, and resource budgeting tools that make each unit of consumption traceable. When the platform exposes these cost signals back to the agents themselves, they can adjust their strategies, optimize their queries, or alter the timing of their work in response. Over time, agents begin to self-regulate based on economic feedback, a capability that is only possible when the data layer provides clear, actionable telemetry. With all of these shifts in application behavior, distributed SQL is the only database paradigm designed to operate at the same level of flexibility and dynamism.

## Infrastructure as a Platform for Doing More

Infrastructure provides the foundation, but a foundation by itself does not complete the structure. Memory systems, distributed databases, and retrieval layers establish the conditions for coherence, scale, and resilience,

yet they remain only potential until harnessed by effective patterns. Distributed SQL plays a central role here, unifying data consistency across stores so that agents can rely on a stable substrate for retrieval and reasoning. Agentic AI is not advanced by storage mechanisms alone, but by the ways those mechanisms are orchestrated into adaptive retrieval strategies that make continuity actionable. Just as roads enable travel only when routes are planned and vehicles know how to navigate them, infrastructure for memory enables intelligence only when patterns guide its use. The next chapter explores these patterns and how agents use them to perceive, reason, act, and learn through infrastructures deliberately designed to support them.

---

1 Edge Delta Team, "What Percentage of Data Is Unstructured? 3 Must-Know Statistics", Edge Delta (blog), March 6, 2024.

2 David Kucher, "AI for Structured Data: Building Production-Ready Agents", Alation (blog), June 4, 2025.

3 Chad DeChant, "Episodic Memory in AI Agents Poses Risks that Should Be Studied and Mitigated", arXiv (preprint), January 20, 2025, arXiv:2501.11739.

4 Deekshith Marla, "Navigating the Trade-Offs: Latency, Cost, and Performance in Agentic Systems", Arya.ai (blog), July 4, 2025.

5 Clément Stenac, "The LLM Mesh: A Common Backbone for Generative AI Applications", Dataiku (blog), September 23, 2023.

# Chapter 3. Beyond Storage: Patterns for Agentic Applications

Chapter 2 framed distributed SQL as more than a storage technology positioned as the backbone of memory systems that unify meaning and truth. Having established this foundation, the next step is to explore how such capabilities are used in practice. Atop the different data stores apps apply patterns, repeatable strategies that apply distributed SQL to the challenges of agentic AI. Each pattern demonstrates how infrastructure becomes intelligence, transforming distributed SQL from a theoretical substrate into a practical framework for adaptive reasoning.

## Beyond Storage: Semantic and Transactional Patterns

With an infrastructure foundation in place, the next step is to examine the most basic query patterns that illustrate how distributed SQL binds meaning and truth together. These patterns show how vectors and transactions, once handled in isolation, can now be joined within a single query fabric. Semantics and facts in distributed SQL are not separate, so they can interact natively, producing results that are both contextually rich and operationally valid. This convergence of SQL and semantic retrieval marks the entry point for practical design for many of the patterns that follow, where retrieval begins to look less like isolated lookups and more like coherent memory. Here are three patterns for how SQL and semantic retrieval work together.

## Semantic-Transactional Join

This pattern combines vector similarity search with relational filters, narrowing results by enforcing factual conditions alongside semantic matches. For example, a support agent might retrieve cases similar to a new ticket but only from active accounts (`WHERE status='active'`). Research on hybrid search shows that such filtering improves precision by excluding semantically relevant but operationally invalid results. In enterprise settings, it strengthens explainability and auditability by grounding semantic reasoning in authoritative facts.[1]

## Contextual Fact Augmentation

In this pattern, embeddings are enriched with structured data rather than filtered by it. For instance, a sales assistant might pair similar past conversations with details like renewal dates, account size, or region. Joining embeddings with these attributes in a distributed SQL query makes an assistant gain both narrative similarity and factual context. Research on retrieval-augmented generation shows that combining unstructured and structured signals improves accuracy and reduces hallucinations.[2, 3]

## Probabilistic Joins for Ambiguous Context

This pattern expands traditional SQL joins with fuzzy or probabilistic matching, which is especially useful when identities or relationships are uncertain. When combined with semantic search, agents can reason over graded confidence levels instead of binary matches. It works this way:

1. Attributes (e.g., name, address) are matched with similarity functions instead of strict equality.
2. Distributed SQL executes joins that produce match probabilities or confidence scores.
3. Embeddings provide semantic context, weighted by probabilistic match scores.

This pattern is powerful in domains like fraud detection, entity resolution, and intelligence analysis, where strict joins fail but probabilistic reasoning is essential. It allows agents to reason under uncertainty instead of discarding partial signals.[4]

Semantic-transactional joins, contextual fact augmentation, and probabilistic joins establish how distributed SQL unifies semantics and transactions, but retrieval alone does not satisfy the demands of agentic AI. Agents must also act on information that is both immediate and contextual, drawing from the present state of the world while recognizing broader patterns over time. This is where processing transactions and analytics in the same system becomes essential. The next section turns to patterns that demonstrate how immediacy and history can be fused into a single decision-making substrate.

## Mixed Workloads: Blending Real-Time Inference with Up-To-Date Context

Agentic AI requires decisions that are both immediate and informed. This dual demand highlights the value of *mixed workload processing*,[5] where online transactional and analytical queries run together in a single distributed SQL system. *Online transaction processing* (OLTP) handles fast, fine-grained transactions. These transactions provide the ground truth of the system; that is, the state of the world at the present moment. *Online analytical processing* (OLAP), by contrast, looks across historical data to identify patterns, trends, and anomalies.

Traditionally, OLTP and OLAP were separated into different systems, with the former used more for rapid writes and the latter used for complex analytical queries. This led to pipelines where operational data was periodically copied into warehouses or lakes, which introduced delays and inconsistencies. Mixed OLAP/OLTP eliminates the separation between transactional and analytical workloads by enabling real-time decision making based on live data rather than delayed, batch-processed insights.

For agentic AI especially, such lags are unacceptable. An agent recommending medical intervention, responding to fraud, or adjusting a supply chain must reason on fresh data with analytical depth.

Distributed SQL databases like TiDB make this possible using Raft-based consensus that ensures transactional consistency, while a columnar engine supports analytical queries.[6] The value of mixed workload processing for agentic AI lies in patterns that actively merge immediacy with perspective. Two such patterns include *sliding window context* and *microbatch refresh*.

## Sliding Window Context

In many agentic scenarios, agents must weigh both the latest event stream and long-term historical behavior. As shown in Figure 3-1, a sliding window provides this dual view by maintaining a moving slice of the most recent data alongside aggregated measures across longer horizons. This enables real-time inference that is tempered by context.

This view lets agentic AI respond to current signals without losing contextual grounding. Technically, most aggregates, such as counts, sums, or averages, can be updated in constant time, while more complex metrics, like percentiles or distinct counts, rely on logarithmic or near-constant updates when implemented with approximate structures such as sketches or t-digests. Memory usage scales with the window size (W) and number of partitions (K), but incremental or approximate methods reduce this dramatically by storing only rolling state rather than all raw events. For vector data, approximate nearest-neighbor indexes combined with time-based eviction keep both retrieval latency and memory predictable. Because updates are incremental and idempotent, failed or delayed batches can be replayed safely without duplication. This makes the sliding window pattern efficient, fault tolerant, and well-suited for real-time inference that balances responsiveness with stability.

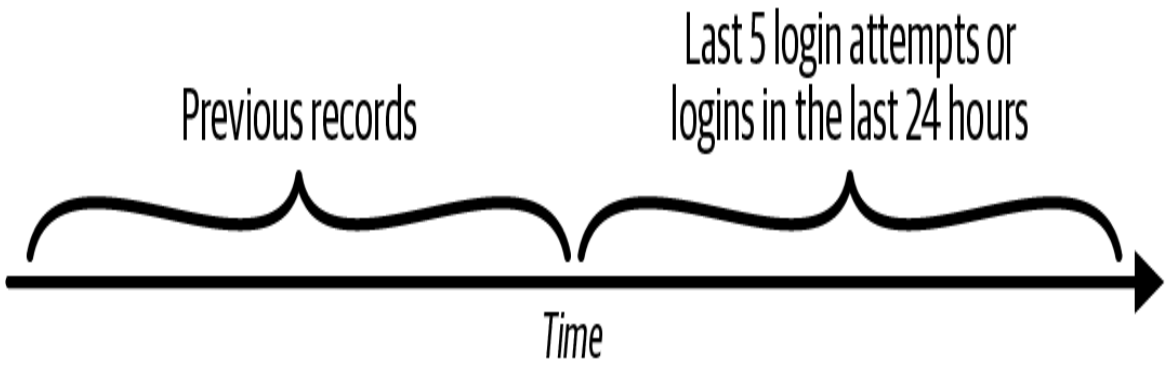


*Figure 3-1. An example sliding window context*

In cybersecurity, a failed login on its own may not appear suspicious. Yet when placed in context, such as dozens of failed attempts from the same IP within minutes or an unusual location compared to historical behavior, the risk becomes clearer. Mixed workload processing supports this by analyzing real-time login events alongside historical aggregates within a single distributed SQL system. Studies on mixed workloads show that this fusion of immediate and long-term data is highly effective for anomaly detection and fraud prevention, where speed and historical grounding must work together.[7]

## Microbatch Refresh

Microbatch refresh addresses the tension between real-time streaming and batch analytics in agentic AI. Instead of continuously updating analytical views (which can overwhelm the system) or relying on large, delayed ETL (extract, transform, load) jobs (which introduce staleness) microbatch refresh processes small increments of new data at frequent intervals, such as every few seconds or minutes, as shown in Figure 3-2.

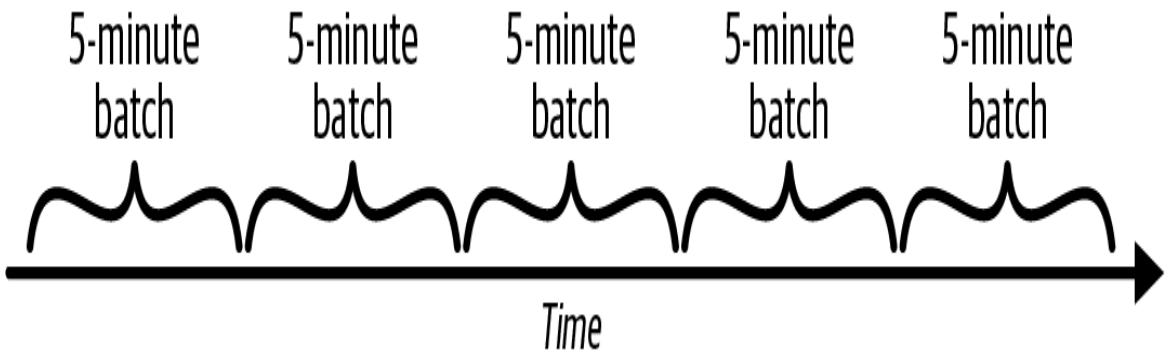


*Figure 3-2. An example microbatch refresh*

In a distributed SQL context, this pattern enables agents to consult near-real-time analytical aggregates that reflect current transactional reality without the full overhead of continuous streaming pipelines. For example, an agent monitoring supply chain risk could query rolling five-minute aggregates of order delays and inventory levels—fast enough for decision making, yet efficient enough to scale globally. The pattern goes as follows:

1. Set the target freshness (e.g., data should be no more than 60 seconds old).
2. Trigger batches on a timer or when enough new rows arrive.
3. Ingest only the new data since the last run, with a small safety delay to avoid race conditions.
4. Validate, dedupe, and transform just the changed records.
5. Merge updates atomically into aggregates, rolling windows, or embeddings.
6. Advance the watermark and log metrics for monitoring.
7. Expose refreshed results (with a timestamp) to agents for reliable, near-real-time retrieval.

When batches fail or are delayed, the system should fall back to the last successful watermark and retry the job with exponential backoff to avoid cascading load. Distributed SQL architectures typically support idempotent merges and atomic upserts, allowing failed batches to be reprocessed safely

without data duplication or corruption. If the delay exceeds the target freshness window, agents can be signaled through stale data flags or health metrics to make degraded, but still coherent, decisions using the most recent confirmed snapshot. This failure-aware design keeps analytical views reliable even under transient faults or network congestion.

The patterns explored so far have focused on building structures that keep data fresh, consistent, and analytically accessible. They ensure that what agents draw upon is timely and trustworthy, reducing the fragility of pipelines and eliminating the lag between transactions and insights. Yet structures alone do not create intelligence. To become useful in agentic applications, these foundations must be paired with retrieval strategies that determine what information is surfaced, how it is contextualized, and when it is recalled. The next set of patterns shifts from maintaining data integrity to activating it, showing how memory is shaped into actionable context for reasoning agents. These basic query patterns underscore the patterns that come next.

# Patterns for Retrieval in Agentic Memory

*Retrieval patterns* define how information is located, recalled, and injected back into reasoning. Just as memory in humans is not only about storage but also about recall, AI systems depend on retrieval strategies to transform stored knowledge into actionable context.

## Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) is an AI architecture that enriches an LLM by enabling it to consult external, authoritative knowledge sources, such as documents, databases, or vector stores, before generating responses. RAG is widely adopted in business settings where domain-specific, policy-driven, or internal information must be accessible through AI systems. A survey by Unisphere Research (in partnership with Semantic Web Company) found that 85% of organizations are either testing or actively

deploying large language models, and nearly one-third (29%) have implemented RAG solutions to bridge corporate databases and LLMs.

While not exclusive to agentic memory, RAG is a pattern that agentic memory uses for practically all kinds of memory in some form, be it short-term, long-term, or episodic. RAG helps integrate knowledge from knowledge bases and memory not stored in relational stores.[8]

As shown in Figure 3-3, RAG involves several stages:

*Indexing*

External data, such as documents, PDFs, images, videos, and websites, is vectorized, a process that uses an embedding model to convert the data into embeddings. The data is stored in a vector database for efficient similarity-based lookup rather than Boolean logic.

*Retrieval*

Upon receiving a user query, a retriever mechanism searches the vector store to identify the most pertinent documents or data snippets. Retrieval uses the same embedding models used by indexing to create an embedding for searching that is compared to the embeddings used for vector databases.

*Augmentation*

The retrieved content is incorporated into the prompt via prompt engineering to enrich the model's context with relevant information.

*Generation*

The LLM generates its response using both its internal parameters and the freshly retrieved external context.

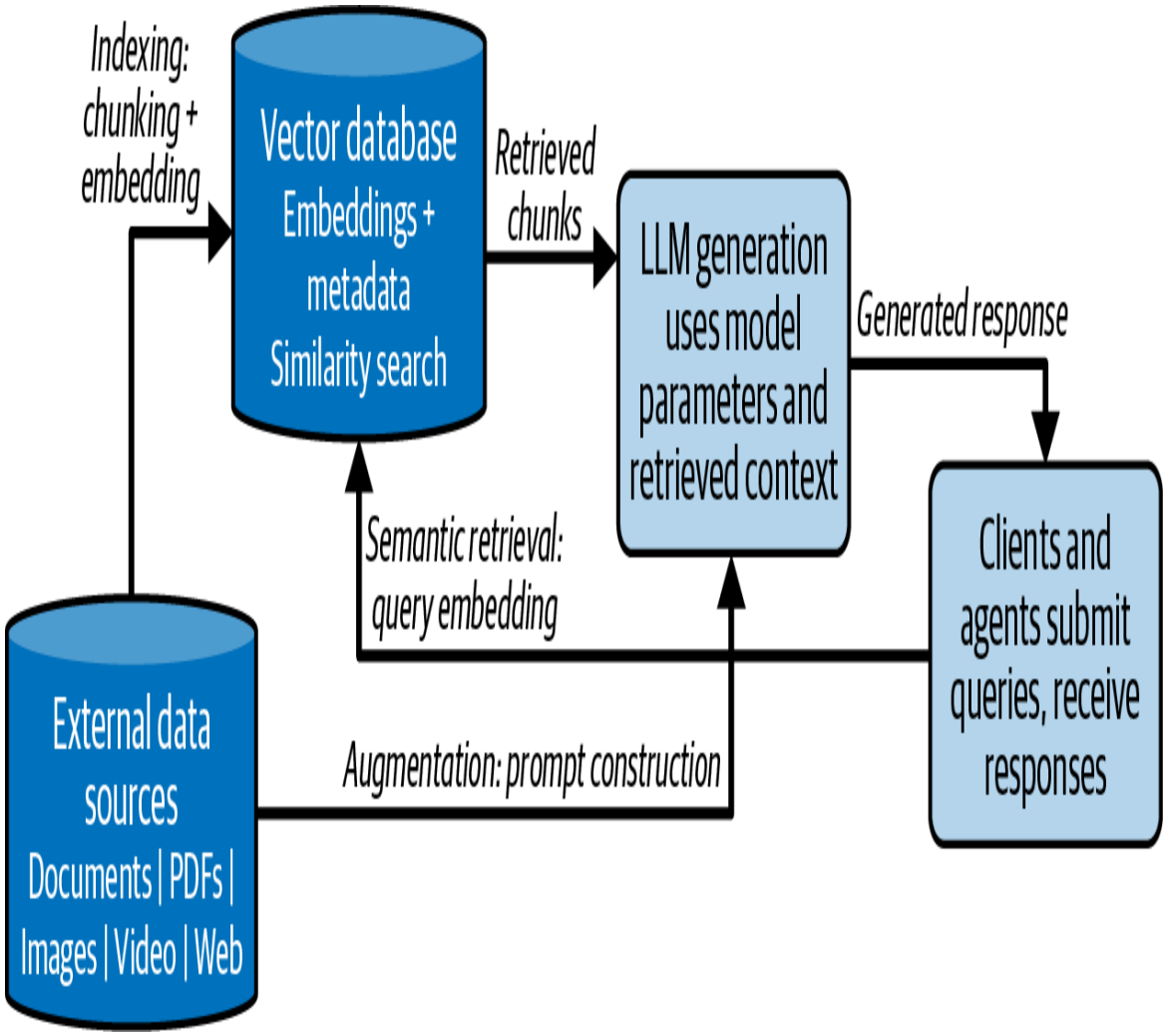


*Figure 3-3. Dataflow in RAG apps*

One of RAG's standout advantages is its ability to stay up to date without retraining the core language model. Instead of investing heavily in retraining or fine-tuning, one can simply update or expand the external knowledge base that the model retrieves from.

Some implementations of RAG can surface citations or references to the retrieved documents, thus giving users visibility into the model's knowledge sources. This added layer of traceability boosts user confidence, as it allows independent verification of the information and helps users understand the basis of the AI's responses.

## Long-Term Memory: Persistent Stores for Agent Continuity

In AI agents, *long-term memory* (LTM) refers to mechanisms that persist meaningful information across user sessions, thus maintaining continuity and personalization over time. LTM is a sort of general memory. Unlike short-term memory, which is limited to the current interaction, LTM allows an agent to remember user-specific facts, like preferences or past instructions, across conversations. Here are the four steps for capturing and storing LTM:

*Capture*

When a conversation ends (or at checkpoints), the system generates a summary or embedding of what was important. This avoids saving everything verbatim.

*Filtering/deciding what to store*

There's no one way that LTM filters and decides what to store, but generally it will use one or more of the following:

- Heuristics to only store information that is important, frequently referenced, or relevant to the user
- Scoring to assign importance scores, like recency, frequency, or explicit user signals to "remember this"
- Forgetting rules to govern how unused memories fade unless reinforced

*Storage*

There's no generally preferred storage for LTM, but many common storage systems are used, including:

- Vector database for semantic recall; for example "what did we talk about regarding pricing?"

- Structured DB/key-value store for explicit facts, such as “user name = Jane, color = dark mode”)
- Multitiered systems with short-term, mid-term, and long-term layers

*Retrieval*

> When the agent runs, it queries memory stores in the same way that RAG queries a vector DB. Retrieved memories—be they summaries, facts, or embeddings— are added to the LLM’s context window so the agent feels “remembering.”

LTM has many functions, but two primary functions include persisting user context and persisting intention in prompts. With user context, an agent with persistent memory adapts naturally to a user’s ongoing preferences and history. This continuity creates smoother, more intuitive experiences that evolve with the user.

Beyond user data, LTM stores past interactions, such as prompts, which can trigger responses like “You asked me to remind you…” The agent demonstrates awareness and foresight, thus making the agent feel smarter and more dependable to the user. The agent becomes an active, thoughtful partner in the interaction, rather than a reactive tool that forgets as soon as the session ends.

Past the immediately realized benefits, memory-equipped agents help eliminate redundancy and focus on what truly matters. They don’t force users to repeat details or context, which allows the agent to concentrate on relevant content and makes interactions leaner, faster, and more effective.

## Episodic Stores: Session-Based, Time-Bound Context Retention

*Episodic memory* refers to mechanisms that preserve the details of specific sessions or interactions, enabling the agent to maintain continuity within a bounded time frame. Episodic memory is a sort of event memory. Unlike long-term memory, episodic memory is tied to a particular conversation or

task, so it captures the flow of what just happened so the agent can stay coherent and responsive.[9]

The pattern in episodic memory avoids repetition and preserves dialogue continuity so multistep reasoning processes stay on track. For example, when a user says, “Continue where we left off,” the agent can retrieve the previous steps and move forward seamlessly. This is accomplished by:

*Capture*

During an active session, the system records events, exchanges, and outcomes. This may include a rolling transcript, structured logs, or embeddings representing the flow of the conversation. Rather than storing generalized knowledge, episodic memory focuses on the specifics of that moment.

*Filtering/deciding what to store*

Episodic memory generally stores more detail than LTM, but it is still selective. Here are three ways of filtering:

- Heuristics keep track of interactions critical to the ongoing task, such as the steps already completed.
- Scoring may weigh importance based on temporal sequence or relevance to the task at hand.
- Session-bound rules are applied once the session concludes: episodic memory often expires or is compressed into a summary for potential transfer into LTM.

*Storage*

Episodic memory is usually temporary and lives in systems designed for short-lived retention, such as:

- Session buffers or rolling windows that track the last N exchanges

- Event logs that can be referenced during a session
- Intermediate stores that may later distill into LTM if the content proves significant

*Retrieval*

When the agent is reasoning during a session, it recalls episodic memory to maintain thread coherence, much like flipping back to an earlier page of a conversation. Retrieved snippets may include step-by-step instructions already given, previous user clarifications, or unresolved tasks. These are injected into the LLM's working context so the agent behaves as if it "remembers" the ongoing episode.

Beyond immediate dialogue flow, episodic memory also helps create an agent that feels attentive and situationally aware. Recalling recent exchanges like "Earlier you asked me to compare options A and B" make the agent appear responsive and grounded in the present interaction. This makes the experience more natural and human-like, even if the memory does not persist beyond the session.

## Temporal Consistency Retrieval

*Temporal consistency retrieval* is a pattern that enables agentic AI to reason on both the current state of facts and the state of facts as they existed at a given time. Distributed SQL systems with temporal tables enable this by maintaining historical versions of records, which can be queried directly. This supports reasoning that is time-sensitive, auditable, and historically faithful. The dataflow generally follows these steps:

1. Data is written into temporal tables that automatically track valid-time or transaction-time history.
2. When retrieving context, the agent includes temporal predicates (e.g., `AS OF TIMESTAMP`) to reconstruct the world at a specific moment.

3. Embeddings tied to historical records are retrieved alongside their temporal validity, ensuring semantic recall is historically anchored.
4. Retrieved facts are passed into the agent's reasoning pipeline, contextualized with time labels to prevent conflating past and present states.

This pattern allows agents to provide historically consistent answers, avoid "time travel hallucinations," and comply with regulatory requirements (e.g., financial audits, medical histories). It grounds reasoning in what was true when, not just what is true now.[10]

## Multiagent Shared Memory

The *multiagent shared memory* pattern uses distributed SQL as a coordinated memory layer for multiple agents. SQL tables hold structured facts, semantic embeddings, and inter-agent commitments. This enables agents to share state safely without race conditions or conflicting updates.

The pattern follows this general flow:

1. A distributed SQL system hosts common tables for agent-to-agent data exchange.
2. Shared embeddings enable agents to discover semantically related contributions from peers.
3. SQL ensures atomic updates to commitments (e.g., locking resources, recording task progress).
4. Agents read shared state and enrich it with semantic recall, enabling coordinated strategies.

This pattern creates a robust substrate for multiagent cooperation, ensuring semantic richness and transactional safety. It prevents agents from "talking past" each other or duplicating effort.[11]

### Incremental Fact Synchronization

*Incremental fact synchronization*, as shown in Figure 3-4, ensures that vector indexes used for semantic search remain aligned with the latest transactional state in SQL.

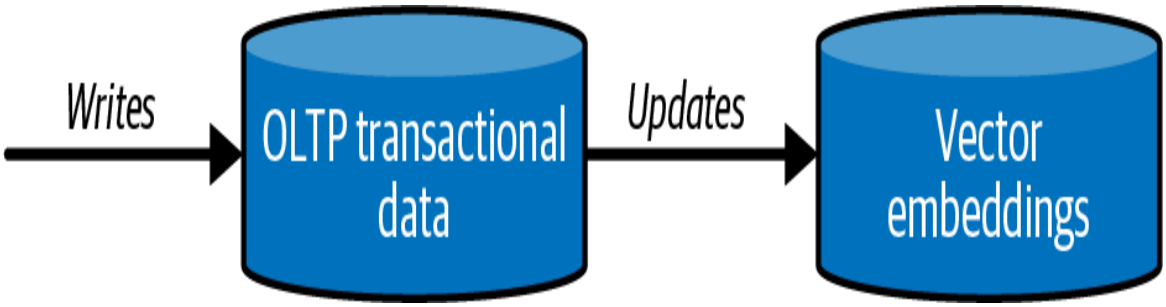


*Figure 3-4. Incremental fact synchronization*

Instead of fragile ETL pipelines, this pattern uses change data capture (CDC) or streaming replication to incrementally update embeddings:[12]

1. Distributed SQL emits row-level changes via CDC or log streams.
2. Updated rows are re-embedded and stored in the vector index.
3. Agents query embeddings that are guaranteed to reflect the current transactional truth.
4. The refreshed embeddings and their transactional facts are provided together during reasoning.

This pattern keeps semantic memory fresh without batch lag, preventing agents from reasoning on stale or contradictory data. It supports real-time alignment between “meaning” (embeddings) and “truth” (transactions).

## Bridging into Oversight

The patterns outlined thus far show how distributed SQL evolves from an infrastructure foundation into a living framework for agentic reasoning. Structures for joining semantics with transactions, handling uncertainty, blending operational immediacy with historical depth, and shaping retrieval

into memory all demonstrate how intelligence emerges from data when access is both timely and trustworthy.

Yet building these capabilities is only part of the story. For distributed SQL to support agentic AI at scale, it must also be administered, monitored, and governed. The next chapter turns to those concerns, focusing on how organizations can oversee distributed SQL systems in ways that ensure reliability, enforce compliance, and provide visibility into the behaviors of agents that depend on them.

---

1 Karthik Menon et al., "Query Attribute Modeling: Improving Search Relevance with Semantic Search and Meta Data Filtering" arXiv, August 6, 2025, arXiv:2508.04683.

2 Patrick Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", *Advances in Neural Information Processing Systems 33*, 2020: 9459–9474.

3 Michael Brenndoerfer, "Hybrid Retrieval: Combining Sparse and Dense Methods for Effective Information Retrieval", November 2, 2025.

4 Tal Friedman and Guy Van den Broeck, "Symbolic Querying of Vector Spaces: Probabilistic Databases Meets Relational Embeddings", in *Proceedings of the 36th Conference on Uncertainty in Artificial Intelligence* (UAI 2020), arXiv, June 27, 2020, arXiv:2002.10029.

5 Edward Funnekotter, "Agentic AI Marks a Turning Point for Business—But Not Without Real-Time Dynamic Access to Business Data", *RTInsights*, June 1, 2025.

6 Dongxu Huang et al., "TiDB: A Raft-Based HTAP Database", *Proceedings of the VLDB Endowment*, 13 (12): 3072–3084, August 1, 2020.

7 TiDB Team, "Real-Time Fraud Detection with TiDB's HTAP Capabilities", PingCAP, March 23, 2025.

8 Patrick Lewis, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks."

9 Joon Sung Park et al., "Generative Agents: Interactive Simulacra of Human Behavior", *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (UIST '23), April 7, 2023, arXiv:2304.03442.

10 Kayla Boggess, Sarit Kraus, and Lu Feng, "Explainable Multi-Agent Reinforcement Learning for Temporal Queries", arXiv, May 17, 2023, arXiv:2305.10378.

11 Guohao Li et al., "CAMEL: Communicative Agents for 'Mind' Exploration of Large Scale Language Model Society", arXiv, March 31, 2023, arXiv:2303.17760.

12 Brian Foster, “Change Data Capture (CDC): A Complete Guide for Modern Data Teams”, PingCAP (blog), October 16, 2025.

# Chapter 4. Operationalizing the AI Memory Layer

In modern AI stacks, applications exercise a variety of retrieval patterns to retrieve context dynamically. Those patterns give the illusion of flexibility, but they also abstract away the robustness of retrieval that cannot be left to application logic. To support agentic AI workloads at scale, the underlying layer must guarantee latency bounds, enforce governance, and ensure relevancy. This chapter shifts focus from "which pattern to choose" to how to operationalize distributed SQL as that reliable substrate. It shows how a distributed SQL database can fulfill the promise behind those retrieval patterns by letting applications simply ask, while the system reliably delivers.

## Latency: Enforcing Speed Under Load

To the application user, retrieval should feel instantaneous, even under heavy concurrency. But applications cannot reliably enforce this. Instead, the infrastructure should guarantee millisecond-level response even as demand scales. Techniques such as index optimization, edge caching, and adaptive prefetching become essential infrastructure strategies. Latency comes in many forms, and each imposes distinct challenges that must be addressed at the system level:

*Best-case (or average-path) latency*

> This is the time a request takes under favorable conditions, such as when caches hit, resources are idle, and the query is simple. It reflects the "happy path" performance users expect most of the time. Ensuring that this stays low requires fast indexing, in-memory access, and minimal internal overhead.

*Tail/worst-case latency*

Even if the median latency is excellent, the slowest responses (e.g., those at the 95th, 99th percentile) can dominate user experience and violate service-level agreements (SLAs)). Outliers can arise from contention, resource interference, or queuing delays. Systems must actively mitigate tail risk via redundancy, speculative execution, scheduling, and resource isolation.[1]

*Cold lookup and cache miss latency*

When a requested item isn't in a fast cache, the system must fall back to deeper storage (disk, remote shards, or index traversal). This "cold path" incurs much higher cost. Infrastructure must optimize the cold path using techniques like prefetching, efficient index structures, and smart caching invalidation policies to avoid large latency cliffs.

*Network/cross-node latency*

In distributed systems, coordination, partitioning, and remote lookups introduce additional delay before a node can access the required data. Even a well-optimized local index can be slowed by cross-node hops. Infrastructure must minimize cross-node calls, colocate related data, leverage locality, or batch requests to reduce this overhead.

*Index maintenance/update latency*

Systems evolve with writes, deletions, reindexing, compaction, or schema changes that must be accounted for. The speed at which these updates propagate without degrading read performance is critical. If index updates are too slow or block reads, they can introduce staleness or spikes in latency. The retrieval substrate must manage incremental updates, background compaction, and consistency without compromising responsiveness.

These latency modes define the real performance envelope. To accomplish this, systems should:

- Embed fast indexes, multimodal unified search, and efficient data structures so both normal and cold-path queries run promptly.
- Control network and coordination latency through partitioning, intelligent routing, and locality awareness.
- Monitor bounding tail latency via speculative execution, isolation, and adaptive resource management.
- Ensure that index update latency (i.e., writes, reindexing, or schema changes) does not degrade read performance.

When that infrastructure works well, applications can assume that everything returns quickly, reliably, and within policy constraints. Instead of each microservice reinventing caching or bulkheads, retrieval becomes a dependable substrate.

## Elasticity and Isolation in Distributed SQL: Why It Matters

Distributed SQL systems are designed to provide a single logical relational interface while distributing storage and compute across many nodes. To serve AI and memory workloads (especially retrieval, embedding lookups, or hybrid queries), these platforms must scale responsively, safely, and without cross-tenant interference. In practice, distributed SQL systems that internalize elasticity and workload isolation avoid fragility in these ways:

*Horizontal scaling on demand*

> Because data is partitioned and replicated across nodes, distributed SQL can scale out when query or indexing load increases and scale back when idle. This elasticity ensures the system can absorb spikes in retrieval traffic without overprovisioning. For example, TiDB splits data into regions and can dynamically redistribute loads to prevent hotspots.

*Resource isolation across tenants or agent workloads*

Just because multiple agents (or users) share the same distributed SQL cluster does not mean they share resources. The system can enforce limits (CPU, memory, query quotas) and isolate workloads so that one tenant or high-volume query doesn't degrade retrieval quality for others. Multitenancy in distributed SQL must go beyond schema isolation by embedding resource-level controls in the query engine and scheduler.

*Partition-aware locality and data coplacement*

To minimize cross-node latency, distributed SQL must be aware of which partitions or shards are frequently accessed together and try to colocate them or at least reduce the number of hops. This reduces coordination costs for queries that join across shards or need multiple context blocks for a retrieval pipeline.

*Elastic compute-storage decoupling*

Many distributed SQL systems decouple compute from storage. This lets the system independently scale query processing nodes (stateless SQL engines) and storage/replica nodes, tuning each to the demands of embedding-based lookups or retrieval fan-outs. This decoupling provides flexibility for the retrieval-heavy paths that can scale their compute cluster aggressively without disturbing the storage layer.

*Safe scaling and versioning with minimal disruption*

When adding nodes, rebalancing shards, or changing schemas, distributed SQL must support rolling upgrades and transparent migration without sacrificing consistency or causing query disruption. In a memory-backed retrieval context, one can't afford "cold gaps" or slowed lookups during rebalancing. The system must handle it gracefully.

*Mitigating cross-tenant "noisy neighbor" behaviors*

In shared distributed systems, sudden heavy index updates or large embedding inserts may overwhelm shared resources. Workload isolation is needed so that such bursts are throttled or sandboxed. This ensures that latency and retrieval quality guarantees for one tenant remain intact even when another is engaged in heavy background operations.

*Multiparadigm storage*

A new generation of distributed SQL databases is emerging to meet the growing convergence of transactional, analytical, and AI-driven workloads. These systems separate compute and storage using cloud native object stores that allow them to scale elastically and maintain strong consistency under fluctuating demand. Beyond traditional SQL, they increasingly support vectors, JSON, and knowledge-graph data to enable richer and more intelligent querying alongside LTM and versioned data storage.[2]

## Putting It Together

An optimized distributed SQL platform with these principles delivers these results:

- Applications can assume that as load scales, the platform will scale elastically to match, without disruption or manual configuration.
- Each agent or tenant can run complex retrievals, embedding lookups, or vector-augmented queries without fear of performance interference.
- Shard locality, coplacement, and isolation reduce cross-node hops and preserve low latency in retrieval paths.
- Upgrades, rebalancing, and dynamic schema changes occur with minimal impact, which preserves high availability and consistency.

In short, leveraging distributed SQL with native elasticity and workload isolation turns what might otherwise be fragile, manually tuned retrieval

architecture into a resilient, production-grade substrate.

## Governance: Baked-In, Not Layered on Top

Access control, compliance, and auditability are often thought of as secondary concerns tacked onto applications through wrappers, middleware, or API gateways. That can work in small-scale settings, but at enterprise scale it becomes fragile, inconsistent, and risky. Instead, governance must be embedded inside the retrieval layer itself, which is (in the case of memory as infrastructure) the infrastructure that handles queries, fetches context, and responds to agents. This is accomplished through several means.

### Standardized Metadata per Request

Every retrieval transaction should carry a consistent set of metadata, such as the request context, data source or shard, timestamp of request, version or revision IDs, user or principal identity, and provenance (how the context was composed). This ensures that downstream the system knows why a retrieval happened, what is being fetched, and where it came from. Without that standardization, audits, debugging, and compliance reviews become brittle and inconsistent.[3]

### Enforced Access Control at the Boundary

Access logic may appear in the application or service layer, but the retrieval layer itself should also enforce access controls. Even if an application has bugs or is compromised, unauthorized data cannot be fetched through the retrieval system. role-based access control (RBAC)) is one such well-understood paradigm for large systems, where users define roles, assign permissions to roles, and then map users or agents to roles, rather than directly managing permissions per user. Moving access enforcement into retrieval centralizes policy, reduces duplication, and avoids policy divergence across many services.

### Immutable Audit Logs for Every Fetch

Every retrieval event—whether successful or denied—should generate an immutable audit log that includes who made the request, what was requested, the metadata, whether access was granted or denied, and timing. Such logs are essential for traceability, compliance, forensic review, and accountability. Unlike regular logs, which are primarily for debugging, audit logs are designed to document "who did what, when, and where" in a nonmodifiable trail.

Embedding these governance mechanisms inside retrieval reduces the risk that applications will diverge in policy or leave security gaps.

## Accuracy: A Continuous, Systemic Responsibility

Applications may attempt to improve relevance by writing custom ranking heuristics, applying filters, or combining signal sources, but true retrieval accuracy (that is, the ability to return the right context while minimizing noise and maximizing relevance) is not something that can reliably live in application code. It is a property of system design and must be treated as such. Achieving it requires continuous measurements and feedback loops.

### Instrumentation and Metrics at the System Level

To make accuracy real and monitored, the system must measure it continuously across real traffic. That means instrumenting metrics like precision, recall, mean reciprocal rank (MRR), normalized discounted cumulative gain (NDCG), and hit rates against ground truth or proxy signals. Without system-level feedback, application logic will drift because models, heuristics, and filters accumulate erosion over time as data shifts, context changes, and domain drift occurs. Embedding instrumentation in the retrieval engine ensures that you can detect when relevance degrades, when certain query patterns underperform, and when new data regimes

break heuristics. This detection is the foundation for proactive adaptation, not reactive patching.

### Infrastructure-Supported Feedback Loops

While measurement is essential, it is just the first step. The real power comes when the system supports feedback loops that convert signals from users, errors, or downstream components into ongoing corrective adjustments. Imagine a system that captures explicit user corrections or relevant judgments (e.g., “relevant/not relevant,” clicks, and star ratings), and routes them into reranker modules or ranking-tuning logic. Simultaneously, the system should detect model disagreements—for example when a generative model declines to use a piece of context, revises its answer, or returns a low confidence—and feed those signals back as supervision, prompting the system to reprioritize or reorder candidate context.[4]

This kind of feedback enables what one might call living indexing. Rather than a static, one-time index, the retrieval engine becomes adaptive, allowing learned improvements over time in ranking weights, scoring fusion strategies, or even periodic embedding updates. Because all of this feedback handling lives inside the infrastructure (not scattered across individual applications), the system can maintain consistent update logic, enforce versioning, allow safe rollback, and avoid duplicated feedback pipelines across services. The result is a retrieval substrate that learns from its mistakes and refines its relevance behavior at scale.

## Applications Consume; Systems Guarantee

In the end, the principle is this: applications consume, but the system guarantees. In a mature architecture, the burden of reliability does not fall on individual services writing ad hoc code to patch latency, security, or relevance gaps. Instead, the retrieval infrastructure itself must deliver on those guarantees. The application’s role is to issue a query while the system’s role is to fulfill it with speed, trust, and meaning. When that

boundary holds firm, then the memory substrate becomes a dependable foundation. Systems become composable, agents can evolve without re-engineering retrieval logic, and trust is baked into the architecture. That shift from fragile application-level heuristics to infrastructure-level guarantees is what transforms memory from a footnote into a first-class backbone of intelligent systems.

---

1 Jeffrey Dean and Luiz André Barroso, "The Tail at Scale", *Communications of the ACM*, 56 (2013): 74–80.

2 Ben Meadowcroft, "Introducing TiDB X: A New Foundation for Distributed SQL in the Era of AI", PingCAP (blog), October 8, 2025.

3 Rocio Aldeco-Pérez and Luc Moreau, "Provenance-Based Auditing of Private Data Use", paper presented at Visions of Computer Science, BCS International Academic Conference, August 2008.

4 Chen Shi et al., "User Feedback and Ranking In-a-Loop: Towards Self-Adaptive Dialogue Systems", in *SIGIR '21: Proceedings of the 44th International ACM SIGI Conference on Research and Development in Information Retrieval*: 2046–2050.

## About the Authors

**Blaize Stewart** is an experienced systems architect with a passion for building scalable, real-time solutions. From writing his first web apps in high school to developing complex applications across embedded systems, handheld devices, desktop apps, and the cloud, Blaize has continuously embraced emerging technologies. His expertise spans cloud-based architectures, IoT solutions, and real-time data streaming. With extensive hands-on experience in building globally scalable systems on Azure and integrating cutting-edge technologies like RAG architectures and vector databases, Blaize is adept at addressing the technical and business challenges of modern AI-driven data pipelines.

**Ed Huang** is the cofounder and CTO of PingCAP, the company behind TiDB, a widely adopted open source distributed SQL database. With over a decade of expertise in distributed systems, database architecture, and cloud computing, Ed applies his deep background in the open source community to solve the challenges of implementing distributed databases in large-scale production environments. He is passionate about streamlining software architecture and helping businesses unlock the full potential of their data.